

Set-Covering mit evolutionären Algorithmen

Zwischenstand und erste Ergebnisse

- Thema: Kostenminimierung unter harter Abdeckungsbedingung
- Vergleich: Hillclimber, (1+1)-EA und genetischer Algorithmus (GA)
- Ziel: fairer Methodenvergleich bei gleichem Evaluationsbudget

1) Was ist das Problem?

- Beim Set-Covering müssen alle Zeilen abgedeckt sein
- Jede gewählte Spalte kostet Geld
- Ziel: minimale Gesamtkosten bei vollständiger Abdeckung
- Darstellung: Bitvektor (Spalte an/aus)
- Gültig ist nur `Uncovered = 0`

2) Wie gehen wir vor?

- Parser liest OR-Library-Instanzen (z. B. data/scp41.txt)
- Gleiche Fitness für alle Verfahren (Reihenfolge): weniger Uncovered -> kleinere Cost -> kleinere Redundancy
- Constraint-Handling per Repair:
 - Greedy Spalten hinzufügen bis alles abgedeckt ist
 - Danach redundante Spalten entfernen
- Runner führt alle Algorithmen über mehrere Seeds aus

3) Welche Algorithmen und wie funktionieren sie?

Algorithmus	Prinzip	Wichtige Parameter
Hillclimber	1-Bit-Nachbar, akzeptiert Verbesserung	PInit, AcceptEqual, Budget
(1+1)-EA	Mehrfach-Bitflip mit pMut, akzeptiert nicht schlechter	PInit, PMut, Budget
GA	Population + Turnier + Crossover + Mutation	PopSize, TournamentSize, CrossoverProb, PMut

- Alle drei laufen mit identischer Fitness und identischem Budget

4) Run-Setup und Ergebnisse

Konfiguration

- Instanz: scp41.txt
- Seeds: 1..5
- Budget: 50_000
- UseRepair = true

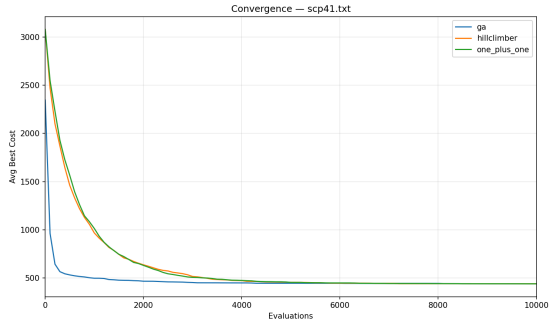
- Bei allen Verfahren: Uncovered = 0

Finale Kosten (gemessener Run)

Algorithmus	Avg	Best	Worst
GA	437.8	435	439
(1+1)-EA	442.0	433	449
Hillclimber	444.0	433	457

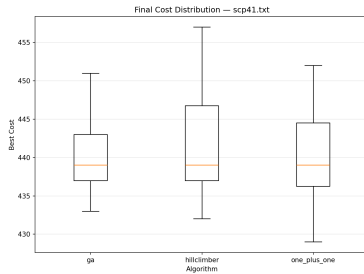
5) Convergence-Plot (0 bis 10k): Was zeigt er?

- Fokus auf die frühe Suchphase mit den meisten Verbesserungen
- Niedriger ist besser; der GA liegt meist vorn



6) Boxplot: Was zeigt die Verteilung?

- Streuung der finalen Kosten über alle Seeds
- GA: beste mittlere Lage und relativ stabil
- HC/(1+1)-EA: teils starke Einzelruns, aber mehr Schwankung



7) Wie wurden die Ergebnisse erreicht?

- Einheitlicher Runner und gleiches Budget für fairen Vergleich
- Seed-basierte Wiederholungen für robustere Aussagen
- Repair hält Lösungen gültig und stabilisiert die Suche
- Logging in `results/raw` macht Ergebnisse reproduzierbar
- `scripts/plot.py` erzeugt die Vergleichsgrafiken

8) Fazit und nächste Schritte

Fazit

- Die End-to-End-Pipeline ist lauffähig
- Auf scp41 ist der GA aktuell im Mittel am besten

Nächste Schritte

- Weitere Instanzen (scp42, scp43, ...) für belastbarere Aussagen
- Mehr Seeds testen (z. B. 30+) und Runs optional parallelisieren
- Kleine Parameterstudie je Algorithmus
- BestKnown pflegen und Gap sauber auswerten
- Externe Config-Datei statt harter Werte in `cmd/main.go`

